

# Software Engineering Design

---

# Topic Covered

---

- Software Design
- Objectives of SW Design
- Principles of SW Design
  - Object-Oriented Design Principles
- Design Pattern
- Software Architecture
- Architectural Design

# Software Design

---

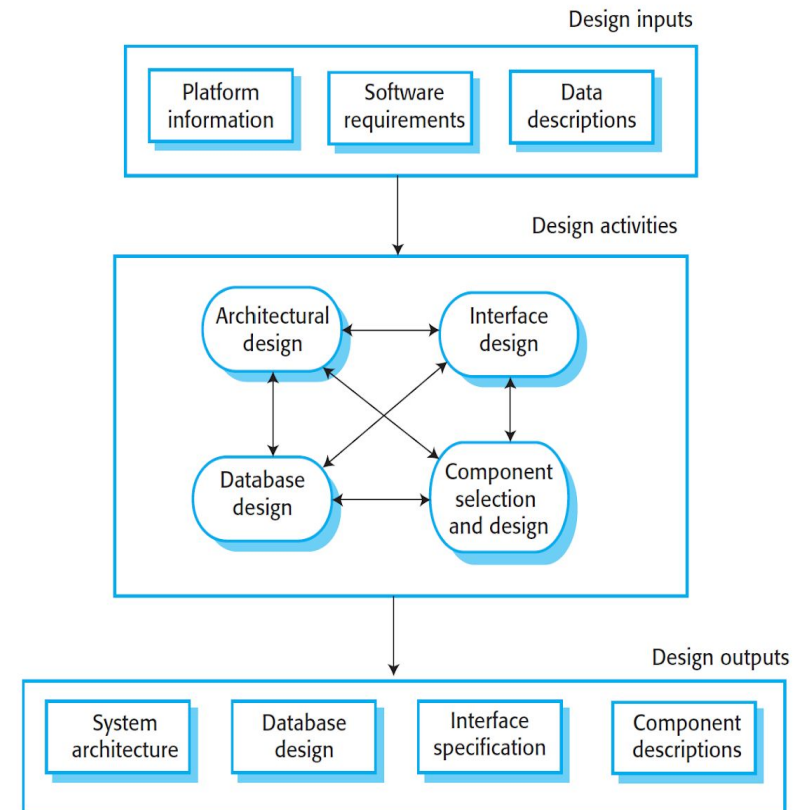
# Software Design

It is the process of transforming requirements into a structured solution that can be implemented in code.

It acts as a **bridge between requirements analysis and implementation.**

The goal is to define **how the system will work** before actually building

**Figure 2.5** A general model of the design process



# Objectives of Software Design

---

# Objectives of Software Design

---

The **objectives of software design** are the **key goals** that guide how software is structured and implemented to meet requirements effectively, efficiently, and reliably.

- **Correctness**
  - Ensure the design accurately implements the specified requirements.
  - The software should behave as intended under all defined conditions.
- **Completeness**
  - Ensures that the design addresses all functional and non-functional requirements. Nothing is left undefined
- **Efficiency**
  - Optimize use of resources such as memory, CPU, and storage.
  - Improve performance (speed, responsiveness) without unnecessary overhead.
- **Maintainability**
  - Make the software easy to understand, modify, and extend.
  - Reduce complexity so that future updates or bug fixes are simpler.

# Objectives of Software Design

---

- Reusability
  - Promote components that can be reused in other projects or modules.
  - Encourage modular design and separation of concerns.
- Flexibility/Extensibility
  - Allow the software to accommodate future changes without major redesign.
  - Support adding new features or modifying existing ones with minimal disruption.
- Consistency
  - There should not be any inconsistency in the design.
  - Avoids conflicting specifications or design elements; the system behaves predictably across modules.
- Reliability
  - Design software to be robust against errors and failures.
  - Ensure consistency, accuracy, and fault tolerance in operation.

# Objectives of Software Design

---

- Usability
  - Ensure that the system is user-friendly and intuitive.
  - Improve interaction with end-users through well-designed interfaces.
- Portability
  - Enable the software to run in different environments with minimal changes.
  - Support various operating systems, hardware, or platforms if required.
- There are more such as scalability, testability, interoperability, simplicity...
- The main objective of software design is to create a system that meets functional requirements while being efficient, maintainable, reliable, and adaptable to future needs.

# Design Principles

---

# Design Principles

---

- A Software Design Principle is a **fundamental guidelines or rules** that help developers design software that is:
  - Maintainable
  - Flexible
  - Scalable
  - Reusable
  - Understandable
- i.e. the principles help achieving the objectives of software design.

# Separation of Concerns (SoC)

---

- Each module should handle one specific concern or responsibility.
- Example:
  - UI logic → View
  - Business logic → Model
  - Request handling → Controller

# Modularity

---

- System should be divided into independent modules.
- **Benefits:**
  - Easier testing
  - Reusability
  - Parallel development
- **Example:**
  - User Module
  - Order Module
  - Payment Module

# Abstraction

---

- Hide unnecessary details and show only essential features.
- Example:
  - User uses:
    - `pay()`
  - But doesn't know:
    - How transaction is processed
    - How encryption works

# Encapsulation

---

- Keep data and methods together and restrict direct access.
- Example:

```
class User {  
    private password;  
    public login();  
}
```

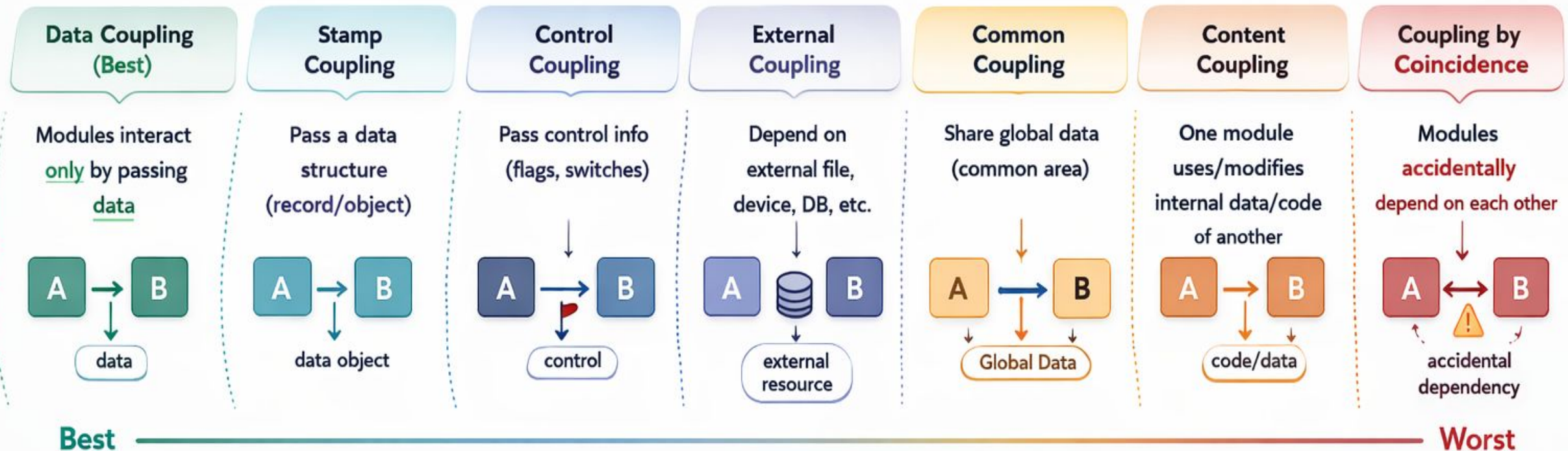
- Password cannot be accessed directly.

# Coupling

---

- Refers to the degree of dependency between modules, classes, or components in a system.
- Modules should have minimal dependency on each other.
- Bad Example:
  - Order module directly modifies Payment module data.
- Good Example:
  - Order calls Payment through an interface.

# Coupling Levels in Software Engineering — Best to Worst



# Cohesion

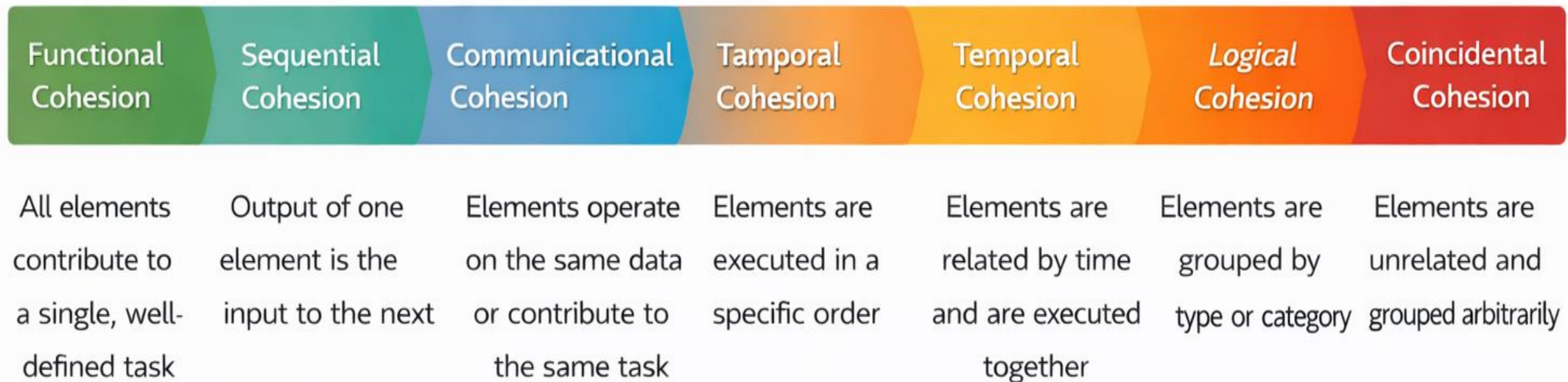
---

- Is the degree to which the elements inside a module, class, or component are related to each other and work together to perform a single task.
- A module should focus on one well-defined task.
- Bad Example:
  - UserModule:
    - Login
    - Payment processing
    - Product listing
- Good Example:
  - UserModule:
    - Only user-related tasks

# Cohesion Levels in Software Engineering

Best

Worst



# Coupling vs. Cohesion

---

| Cohesion                                     | Coupling                                      |
|----------------------------------------------|-----------------------------------------------|
| Measures relationship <b>within a module</b> | Measure the dependency <b>between modules</b> |
| Focuses on internal structure                | Focuses on inter-module interaction           |
| <b>Goal: High Cohesion</b>                   | <b>Goal: Low Coupling</b>                     |
| Improves maintainability and readability     | Improves flexibility and scalability          |

# SOLID principles (Object Oriented Design)

---

The **SOLID principles** are five foundational design guidelines in object-oriented programming that help developers create software that is **easy to maintain, extend, and understand**. They promote **clean, modular, and flexible code**.

## **S — Single Responsibility Principle (SRP)**

- **Definition:** A class should have **only one reason to change**; it should have just one responsibility.
- **Meaning:** Each class or module should focus on a single functionality.
- **Benefit:** Easier to understand, test, and modify without affecting unrelated behavior.

## **O — Open/Closed Principle (OCP)**

- **Definition:** Software entities (classes, modules, functions) should be **open for extension but closed for modification**.
- **Meaning:** You should be able to add new functionality by adding new code, **not by changing existing code**.
- **Benefit:** Makes your system more robust and less prone to bugs from changes.

# SOLID principles

---

## L — Liskov Substitution Principle (LSP)

- **Definition:** Objects of a superclass should be **replaceable with objects of its subclasses** without affecting the correctness of the program.
- **Meaning:** Subclasses must behave in ways consistent with their base classes.
- **Benefit:** Ensures that derived classes extend functionality without breaking existing code.

| Parent and child                                                                           | LSP Violation                                                                                              | Better Design                                                                                       |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <pre>class Bird {<br/>    void fly() {}<br/>}<br/><br/>class Penguin extends Bird {}</pre> | <pre>// Passing a Penguin object will break<br/>the code below as penguin cannot fly<br/>bird.fly();</pre> | <pre>interface Bird {}<br/><br/>interface FlyingBird extends Bird {<br/>    void fly();<br/>}</pre> |

# SOLID principles

---

## I — Interface Segregation Principle (ISP)

- **Definition:** Clients should not be forced to depend on interfaces they do not use.
- **Meaning:** Prefer many **small, specific interfaces** rather than one large, general-purpose interface.
- **Benefit:** Reduces side effects and increases flexibility and clarity.

## D — Dependency Inversion Principle (DIP)

- **Definition:**
  - High-level modules should **not depend on low-level modules**. Both should depend on abstractions (e.g., interfaces).
  - Abstractions should not depend on details. Details should depend on abstractions.
- **Meaning:** Depend on interfaces or abstract classes rather than concrete implementations.
- **Benefit:** Decouples components, making them easier to replace and test.

# SOLID principles

| Principle | Description                                 | Key Idea                         |
|-----------|---------------------------------------------|----------------------------------|
| SRP       | One class, one responsibility               | Single reason to change          |
| OCP       | Open for extension, closed for modification | Extend without modifying         |
| LSP       | Subtypes must be substitutable              | Derived classes behave correctly |
| ISP       | Many specific interfaces                    | Clients use only what they need  |
| DIP       | Depend on abstractions                      | High-level depends on interfaces |

# Design Pattern

---

# Design Pattern

---

- A general, reusable solution to a recurring problem in software design.
- Not a finished piece of code, but a template or blueprint that guides developers in structuring their system effectively.
- Design patterns capture best practices that have been proven successful over time.
- Design patterns help to:
  - Improve code reusability
  - Enhance maintainability
  - Promote scalability
  - Reduce design complexity
  - Improve communication among developers (shared vocabulary)
  - Follow established software engineering principles

# Design Pattern

---

- It is popularized by Gang Of Four( 1994 book)
- Related to Object-Oriented Design
- 3 types
  - Creational Pattern
    - Deal with object creation mechanisms
  - Structural Pattern
    - Deal with relationships between classes and objects.
    - How do objects combine with other object and formalize bigger entity?
  - Behavioral Pattern
    - Deal with communication between objects.

# Creation Pattern

---

- Deal with object creation mechanisms.
- Make a system independent of how its objects are created, composed, and represented.
- There are five creational design patterns:
  - Singleton
  - Factory Pattern
  - Abstract Factory
  - Builder
  - Prototype

# Singleton

---

- Ensures a class has only one instance and provides a global access point to it.
- Where to use
  - Some resources should not have multiple instances:
    - Database connection
    - Logger
    - Configuration manager

# Singleton

---

- How?
  - Make constructor private (or controlled).
  - Store a static instance.
  - Provide a method to access that instance.

- Example

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

# Singleton

---

- Advantages
  - Controlled access
  - Saves resources
  - Ensures consistency
- Disadvantages
  - Hard to test
  - Can introduce global state issues

# Factory Pattern

---

- Defines an interface for creating an object, but lets subclasses decide which class to instantiate.
- Where to use
  - Object creation depends on conditions.
- How?
  - Create a factory method:

# Factory Pattern

- Assume we are designing an app for a company that provides online service of household equipment
- The method that handle this functionality is `processServiceRequest`
- Question: What if there is a new service type??
- Solution: Use Factory Patter

```
1 public void processServiceRequest(String serviceType) {  
2     OnlineService service;  
3     if (service.equals("AirConditioner"))  
4         service = new ACService();  
5     else if (service.equals("WashingMachine"))  
6         service = new WMService();  
7     else if (service.equals("Refrigerator"))  
8         service = new RFService();  
9     else  
10        service = new GeneralService();  
11  
12    service.getinfo();  
13    service.assignServiceRequest();  
14    service.assignAgent();  
15    service.processRequest();  
16 }
```

# Factory Pattern

---

- Remove the code that creates instances to a factory class
- A factory encapsulates object creation logic.

```
1 public class OnlineServiceFactory {  
2     public OnlineService getOnlineService(String type) {  
3         OnlineService service;  
4         if(service.equals("AirConditioner"))  
5             service = new ACService();  
6         else if(service.equals("WashingMachine"))  
7             service = new WMService();  
8         else if(service.equals("Refrigerator"))  
9             service = new RFService();  
10        else  
11            service = new GeneralService();  
12        return service;  
13    }  
14 }
```

# Factory Pattern

---

- Refactor the previous method as shown here
- Benefit: service type changes won't affect the code

```
public void processServiceRequest(String serviceType) {  
    OnlineService service = new OnlineServiceFactory().getOnlineService(serviceType);  
    service.getinfo();  
    service.assignServiceRequest();  
    service.assignAgent();  
    service.processRequest();  
}
```

# Structural Pattern

---

| Pattern   | Purpose                                                |
|-----------|--------------------------------------------------------|
| Adapter   | Converts one interface into another.                   |
| Decorator | Adds behavior dynamically.                             |
| Composite | Treats individual and group objects uniformly.         |
| Facade    | Provides a simplified interface to complex subsystems. |
| Proxy     | Controls access to another object.                     |
| more...   |                                                        |

# Behavioural Pattern

---

| Pattern         | Purpose                                   |
|-----------------|-------------------------------------------|
| Observer        | Notifies dependent objects automatically. |
| Strategy        | Selects algorithms at runtime.            |
| Command         | Encapsulates request as an object.        |
| State           | Changes behavior based on internal state. |
| Template Method | Defines algorithm skeleton.               |
| more...         |                                           |

# Dependency Injection

---

- Dependency Injection (DI) is a design technique where an object does not create its own dependencies, but instead receives them from the outside.

```
class Engine {  
    void start() { }  
}
```

```
class Car {  
    private Engine engine = new Engine(); // Car creates dependency  
}
```

# Dependency Injection

---

- Problems:
  - Car is tightly coupled to Engine
  - Hard to replace Engine
  - Hard to test (can't mock Engine)
  - Violates Single Responsibility Principle

# With Dependency Injection (Loose Coupling)

---

```
class Car {  
    private Engine engine;  
  
    Car(Engine engine) { // Injected from outside  
        this.engine = engine;  
    }  
}
```

- Now:
  - Car doesn't decide which engine to use
  - You can pass PetrolEngine, DieselEngine, or MockEngine
  - Easier testing
  - More flexible design

# Types of Dependency Injection

---

- Constructor Injection (Recommended)
- Setter Injection
- Field Injection (Framework-based)

# Constructor Injection

---

```
class UserService {  
    private EmailService emailService;  
  
    UserService(EmailService emailService) {  
        this.emailService = emailService;  
    }  
}
```

- Immutable
- Required dependency
- Easy to test
- Preferred in most modern systems

# Setter Injection

---

```
class UserService {  
    private EmailService emailService;  
  
    void setEmailService(EmailService emailService) {  
        this.emailService = emailService;  
    }  
}
```

- Optional dependencies
- Object can exist in incomplete state

# Field Injection

---

- **Field Injection** is a type of Dependency Injection where dependencies are injected **directly into class fields**, usually by a framework, without using constructors or setters.

```
class UserService {  
    private EmailService emailService;  
}
```

# Software Design

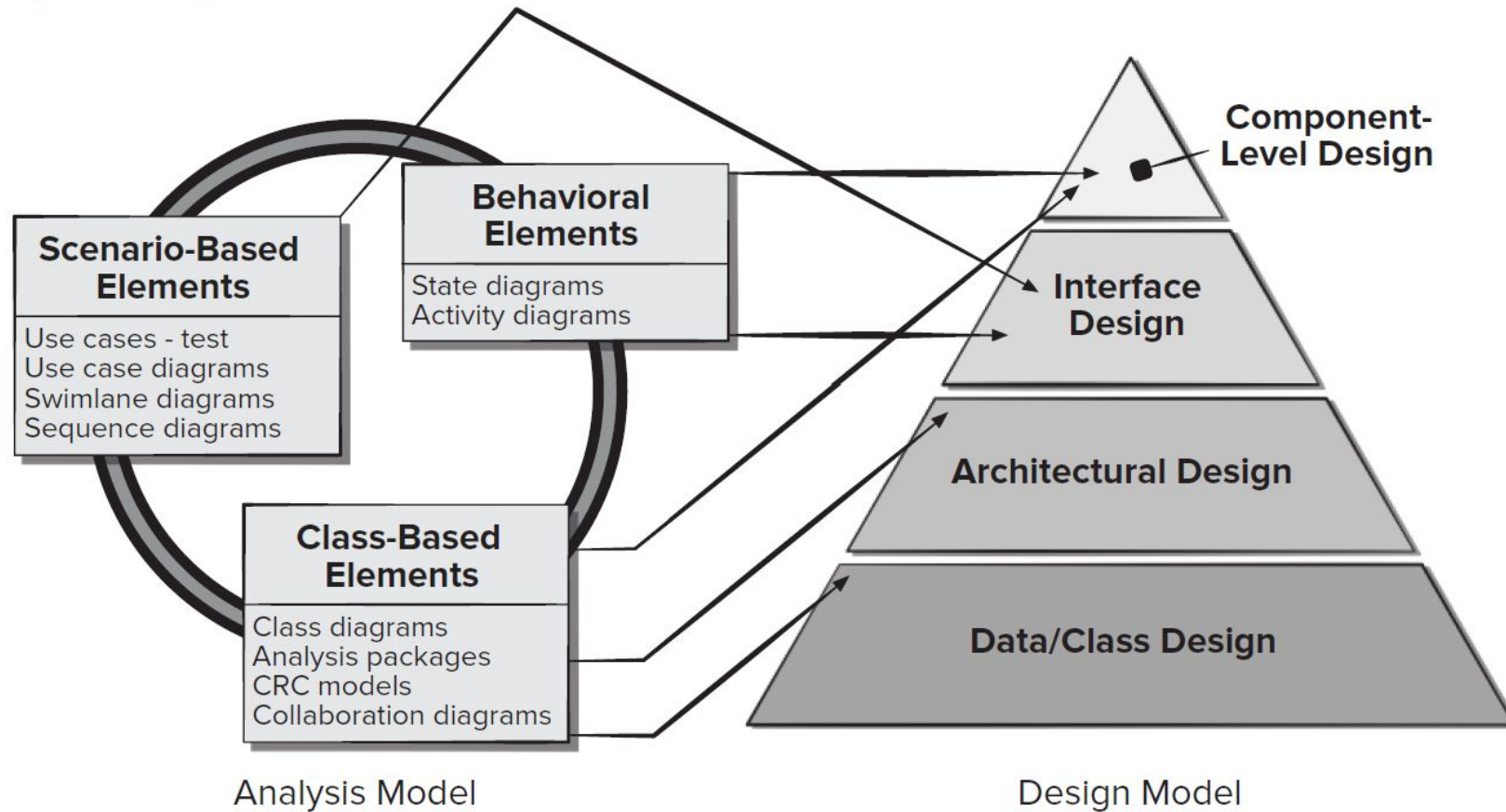
---

# Steps of Software Design

---

- First, the **architecture** of the system or product must be modeled. (Architectural Design)
- Then, the **interfaces** that connect the software to end users, to other systems and devices, and to its own constituent components are represented. (High Level Design)
- Finally, the **software components**, that are used to construct the system, are designed. (Detailed/Low-Level Design)

**FIGURE 9.1** Translating the requirements model into the design model



# Software Architecture

---

# Software Architecture

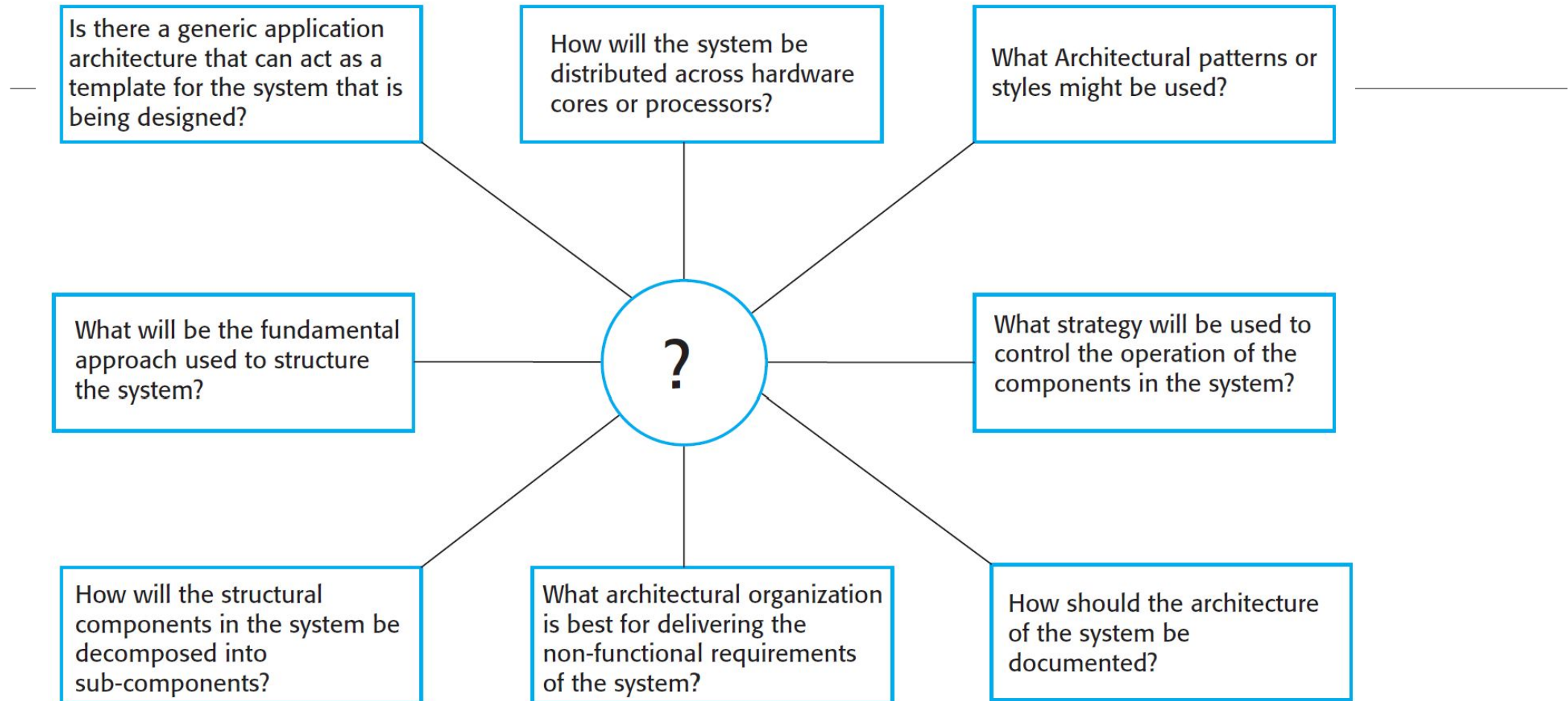
---

- Architectural design is a process of identifying
  - the main structural components in a system
  - how the components should be organized in a software system
  - i.e. designing the overall structure of that system.
- Output → architectural model
  - Describes how the system is organized as a set of communicating components.
  - This is like creating the blueprint of a building.

# Architectural Design

---

- A creative process to design a system organization that will satisfy the functional and non-functional requirements of a system.
- There is no formulaic architectural design process.
- It depends on
  - the type of system being developed,
  - the background and experience of the system architect, and
  - the specific requirements for the system.
- Is based on a **series of decisions** to be made **rather than a sequence of activities**



**Figure 6.2** Architectural design decisions

# Software Architecture

---

- The architecture of a software system may be based on a particular **Architectural pattern or style**.
  - Monolithic
  - Client-server
  - Microservices
  - Layered architecture
  - Event-driven architecture.
  - MVC architecture

# Software Architecture

---

- Software architecture is important because it affects the performance, robustness, distributability, and maintainability (non-functional requirements)
- Because non-functional system characteristics are closely tied to software architecture, the architectural style and structure should be chosen based on the system's non-functional requirements.
  - **Performance:** If performance is a critical requirement, the architecture should be designed
    - to localize critical operations within a small number of components,
    - with these components deployed on the same computer rather than distributed across the network
    - Using large components reduces the number of component communications, as most of the interactions between related system features take place within a component.

# Software Architecture

---

- Security: If security is a critical requirement,
  - Use a layered structure for the architecture
  - with the most critical assets protected in the innermost layers and a high level of security validation applied to these layers.
- Safety: If safety is a critical requirement,
  - the architecture should be designed so that safety-related operations are co-located in a single component or in a small number of components.

# Software Architecture

---

- Availability: If availability is a critical requirement,
  - the architecture should be designed to include redundant components so that it is possible to replace and update components without stopping the system.
- Maintainability: If maintainability is a critical requirement,
  - the system architecture should be designed using fine-grain, self-contained components that may readily be changed. Producers of data should be separated from consumers, and shared data structures should be avoided.

# Software Architecture View

---

- 4 +1 view model of software architecture
- four fundamental architectural views → linked through common use cases or scenarios (+1 view)
  - logical view
  - process view
  - development view
  - Physical view

# Software Architecture View

---

- logical view
  - shows the key abstractions in the system as objects or object classes.
- process view
  - shows how, at run time, the system is composed of interacting processes
  - useful for making judgments about non-functional system characteristics such as performance and availability.

# Software Architecture View

---

- development view
  - shows how the software is decomposed for development.
  - useful for software managers and programmers.
- Physical view
  - shows the system hardware and how software components are distributed across the processors in the system
  - useful for systems engineers planning a system deployment.

# High Level Design

---

Breaks the 'single entity multiple component' concept of architectural design into less abstracted view of sub systems and modules and depicts their interaction with each other.

# Data Design

---

This step focuses on designing how data will be stored and managed.

## **Activities:**

- Identify entities and relationships
- Design database schema
- Create ER diagrams
- Define data structures
- Plan indexing and optimization

## **Outputs:**

- Database schema
- Data dictionary
- Data flow diagrams
- Good data design improves performance and maintainability.

# Interface Design

---

Defines how different components interact.

## **Types of Interfaces:**

- User Interface (UI)
- System Interface (between modules)
- External Interface (APIs, third-party services)

## **Activities:**

- API design
- UI wireframes/mockups
- Define input/output formats
- Error handling design

# Interface Design

---

## Outputs:

- API documentation
- UI prototypes
- Interface contracts
- This ensures smooth communication between parts of the system.

# Component-Level Design (Low-Level Design – LLD)

---

This step defines the internal logic of each module.

## **Activities:**

- Define classes and methods
- Create UML diagrams (class, sequence, activity diagrams)
- Define algorithms
- Specify data structures

# Component-Level Design (Low-Level Design – LLD)

---

## Principles Used:

- SOLID principles
- Design patterns
- Low coupling, high cohesion
- Encapsulation and abstraction

## Outputs:

- Detailed design document
- Pseudocode
- Class diagrams
- This step tells developers exactly how to implement each module.

# Summary of Design Steps

| Step                | Purpose                  |
|---------------------|--------------------------|
| Requirements Review | Understand what to build |
| Architecture Design | Define system structure  |
| Data Design         | Plan data storage        |
| Interface Design    | Define interactions      |
| Component Design    | Detail module logic      |
| Algorithm Design    | Solve specific problems  |
| Security Design     | Protect the system       |
| Verification        | Validate the design      |
| Documentation       | Record decisions         |

# Design Documentation

---

All design decisions are documented clearly.

## **Includes:**

- Architecture diagrams
- UML diagrams
- Database schema
- API specifications
- Design rationale

## **Good documentation helps:**

- Developers understand the system
- New team members onboard easily
- Maintain long-term system quality

---

Thank You

